Version 0.1 — Exploratory Draft

Exploratory Research Document

Instance Space Analysis (ISA) and MATILDA

Methodology, Pipeline and Output Interpretation

Part of the Master Thesis Research

Industrial Engineering
School of Engineering and Applied Sciences
Universidad de los Andes

Lucas Zamora F.

April 15, 2026

Lucas Zamora F.
lzamora@miuandes.cl

Contents

1 Introduction **4**

2 Background: Instance Space Analysis (ISA) **4**

2.1 Motivation and Limitations of Traditional Benchmarking 4

2.2 Conceptual Framework 4

2.3 Interpretation of the ISA Framework 6

2.4 Instance Space Representation 6

2.5 Algorithm Footprints 7

2.6 Insights Enabled by ISA 7

Lucas Zamora F.
lzamora@miuandes.cl

List of Figures

1 Summary of the Instance Space Analysis (ISA) framework, reproduced from [1]. 5

Lucas Zamora F.
lzamora@miuandes.cl

List of Tables

Lucas Zamora F.
lzamora@miuandes.cl

1 Introduction

This document supports the exploratory stage of understanding Instance Space Analysis (ISA), MATILDA, and the interpretation of their outputs. Its purpose is to consolidate concepts, methodology, graphical interpretations, and implementation details relevant to the thesis work.

2 Background: Instance Space Analysis (ISA)

2.1 Motivation and Limitations of Traditional Benchmarking

Traditional approaches to algorithm benchmarking rely on evaluating performance over a fixed set of test instances and reporting aggregate statistics, such as average performance. However, this approach presents significant limitations. In particular, the conclusions drawn are highly dependent on the choice of test instances, which are often inherited from the literature without rigorous assessment of their diversity or representativeness.

As highlighted in [1], two critical issues arise: (i) the lack of mechanisms to evaluate whether benchmark instances are sufficiently diverse and unbiased, and (ii) the inability to understand how algorithm performance varies with respect to instance characteristics. As a result, important insights regarding algorithm strengths and weaknesses remain hidden when relying solely on aggregated performance metrics.

Instance Space Analysis (ISA) addresses these limitations by providing a framework that explicitly models the relationship between instance characteristics and algorithm performance.

2.2 Conceptual Framework

The ISA methodology extends the classical Algorithm Selection Problem introduced by Rice in [2], where the goal is to select the best-performing algorithm for a given problem instance based on its features.

Formally, ISA considers six fundamental components:

- A problem space $\mathcal{P}$, representing all possible instances of a problem domain.
- A subset of instances $\mathcal{I} \subset \mathcal{P}$ for which empirical data is available.
- A feature space $\mathcal{F}$, where each instance $x \in \mathcal{I}$ is represented by a feature vector $\mathbf{f}_x \in \mathbb{R}^m$.
- An algorithm space $\mathcal{A}$, containing the set of candidate algorithms.

Lucas Zamora F.
lzamora@miuandes.cl

- A performance space $\mathcal{Y}$, where each value $y_{\alpha,x}$ measures the performance of algorithm $\alpha \in \mathcal{A}$ on instance x.
- A meta-data set consisting of the matrices

$$\mathbf{F} \in \mathbb{R}^{m \times n}, \qquad \mathbf{Y} \in \mathbb{R}^{a \times n},$$

  where n is the number of instances and a is the number of algorithms.

Within the classical framework, this meta-data is used to learn a selection mapping that predicts the best algorithm for a given instance. ISA extends this paradigm by introducing a new objective: the construction of a low-dimensional representation of the instance space that enables visualization and interpretation of algorithm behavior.

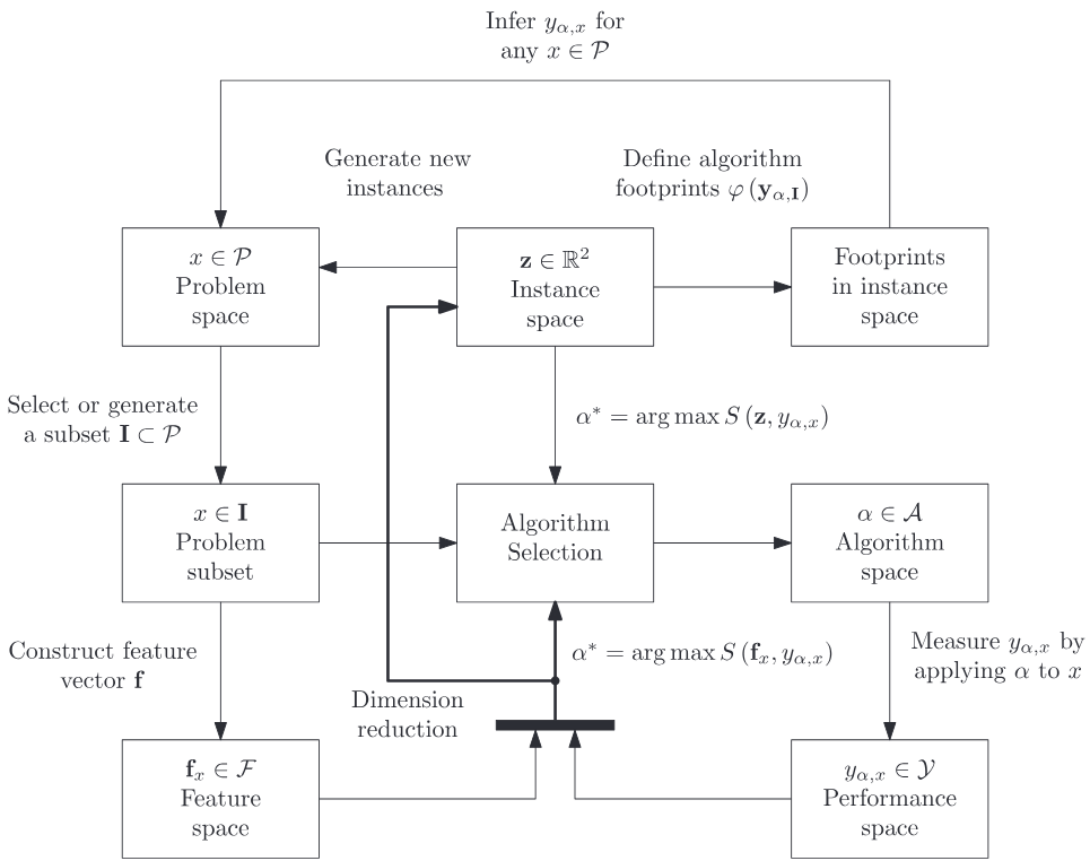

Figure 1: Summary of the Instance Space Analysis (ISA) framework, reproduced from [1].

2.3 Interpretation of the ISA Framework

Figure 1 presents a conceptual overview of the Instance Space Analysis (ISA) framework. The diagram illustrates how problem instances, their features, and algorithm performance are interconnected within a unified analytical structure.

The process begins with the *problem space* $\mathcal{P}$, which represents the universe of all possible problem instances. From this space, a subset $\mathcal{I} \subset \mathcal{P}$ is selected or generated for empirical analysis. Each instance $x \in \mathcal{I}$ is then characterized by a feature vector $\mathbf{f}_x \in \mathcal{F}$, defining its position in the feature space.

A key step in ISA is the transformation of the feature space into a low-dimensional instance space, enabling visualization and supporting the interpretation of algorithm behavior.

Within the instance space, algorithm performance is analyzed through the concept of *algorithm selection*. Given a point $\mathbf{z}$, the framework aims to identify the algorithm $\alpha^* \in \mathcal{A}$ that maximizes a performance measure $S(\cdot)$, effectively linking instance characteristics with expected algorithm behavior.

The *performance space* $\mathcal{Y}$ captures the empirical results obtained by applying each algorithm $\alpha \in \mathcal{A}$ to instances $x \in \mathcal{I}$. These measurements provide the foundation for evaluating and comparing algorithm performance across different regions of the instance space.

An important extension introduced by ISA is the notion of *algorithm footprints*, which represent regions of the instance space where a given algorithm performs well. These footprints allow for a geometric interpretation of algorithm strengths and weaknesses, enabling a deeper understanding of performance variability.

Finally, the framework highlights an iterative feedback loop. Insights obtained from the instance space can be used to generate new problem instances, refine feature definitions, or improve algorithm selection strategies. This iterative process is fundamental to constructing a representative and informative instance space.

2.4 Instance Space Representation

A central contribution of ISA is the projection of instances from the high-dimensional feature space $\mathcal{F}$ into a two-dimensional instance space. Each instance x is mapped to a point $\mathbf{z} \in \mathbb{R}^2$, such that geometric relationships reflect similarities in instance characteristics and algorithm performance.

This projection is constructed to preserve meaningful structure, enabling the identification of regions associated with different levels of difficulty for algorithms. As a result, the instance space provides a visual and analytical tool to explore how algorithm performance varies across different types of instances.

2.5 Algorithm Footprints

Within the instance space, ISA introduces the concept of *algorithm footprints*, defined as regions where an algorithm is expected to perform well based on empirical evidence.

A footprint is characterized by several properties:

- **Location**: indicates the types of instances where the algorithm performs well.
- **Area**: reflects the robustness of the algorithm across the instance space.
- **Density**: measures the strength of supporting empirical evidence.
- **Purity**: quantifies the consistency of performance within the region.

This representation allows for a more nuanced comparison between algorithms, revealing complementary strengths and weaknesses that are not observable through average performance metrics.

2.6 Insights Enabled by ISA

By combining feature information, performance data, and geometric representation, ISA enables:

- Identification of regions where specific algorithms dominate.
- Detection of biases or gaps in benchmark instance sets.
- Understanding of how instance characteristics influence algorithm performance.
- Support for automated algorithm selection through learned mappings.

Furthermore, ISA is inherently iterative: the analysis of the instance space may reveal the need for additional instances, features, or algorithms, leading to progressively more informative representations.

Lucas Zamora F.
lzamora@miuandes.cl

References

[1] KATE SMITH-MILES and MARIO ANDRÉS MUÑOZ. "Instance Space Analysis for Algorithm Testing: Methodology and Software Tools." In: *ACM Computing Surveys* 55.12 (Dec. 2023), pp. 1–31. ISSN: 0360-0300. DOI: 10.1145/3572895. (Visited on 04/15/2026).

[2] "The Algorithm Selection Problem". In: *Advances in Computers*. Vol. 15. Elsevier, Jan. 1976, pp. 65–118. DOI: 10.1016/S0065-2458(08)60520-3. (Visited on 04/15/2026).